

PART III — BUILDING THE RETRIEVAL PIPELINE

Chapter 9

Vector Databases

“A vector database does not store meaning — it stores numbers, and trusts that you embedded them well enough that nearness in space is nearness in meaning.”

Chapter 9 — Vector Databases

Chapter 8 ended with a stream of 384-dimensional vectors coming out of an embedding model — correct, normalized, and completely useless on their own. A list of floats sitting in a Python variable cannot answer a question; it can only be compared to other lists of floats, and only quickly if it is stored in something built for that one job. That something is a vector database, and this chapter is about understanding what it actually does under the hood, choosing one for your situation, and building the `VectorStore` class that the rest of this book's retrieval pipeline will depend on.

By the end of this chapter you will be able to explain precisely what gets stored in a vector record, choose the right distance metric for a given embedding model, justify ChromaDB as the right tool for local development while knowing exactly when to outgrow it, organize data with collections and metadata filters, persist and reload a store across process restarts, and write a batched insertion routine that will not crash your machine on a 50,000-chunk corpus. You will also learn the discipline of backing up and versioning an index — the unglamorous part of the job that saves you when a re-embedding run goes wrong.

The running example throughout is the same one used in the rest of this book: a self-learning, agentic Retrieval-Augmented Generation system built on ChromaDB, sentence-transformers, and LangGraph. Every code sample below is drawn from, or directly compatible with, the `VectorStore` and `EmbeddingManager` classes used in that project.

9.1 What a Vector DB Actually Stores

Strip away the marketing language and a vector database is a system purpose-built to do one operation extremely well: given a query vector, return the k stored vectors that are closest to it, as fast as possible. Everything else — collections, metadata filters, persistence, replication — exists in service of that single operation.

Each record the store holds is a small bundle of four things, kept together as one unit so that a similarity search can return something a human (or an LLM) can actually use, not just a bare list of numbers.

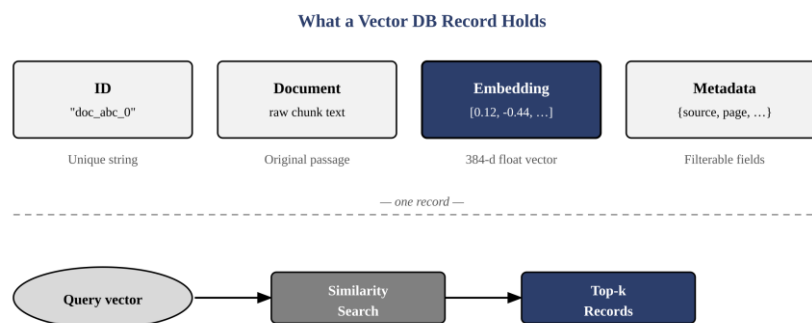


Figure 9.1 — Each record is a four-field tuple; queries return the k nearest by embedding distance.

Figure 9.1 — Each record is a four-field tuple; queries return the k nearest by embedding distance.

The four fields, concretely, in the ChromaDB API used throughout this book:

- **id** — a unique string identifying the record. You assign this; ChromaDB will not generate one for you unless you ask. In the `VectorStore` class built later in this chapter, IDs are constructed from a UUID plus an index, so they remain unique even across repeated ingestion runs.
- **document** — the original chunk of text. This is what gets handed to the LLM as retrieved context. The store keeps it alongside the vector purely for convenience; the vector itself carries no human-readable information.

- **embedding** — the dense vector produced by the embedding model from Chapter 8. Its dimensionality is fixed by the model (384 for all-MiniLM-L6-v2) and must be identical for every record in a collection.
- **metadata** — a flat dictionary of additional fields: source file name, page number, chunk index, ingestion timestamp, document type, and anything else useful for filtering later. Section 9.5 covers the constraints on what this dictionary may contain.

Definition — Vector record

A single stored unit consisting of a unique id, the embedding vector, the source document text, and an optional metadata dictionary. A vector database is, structurally, just a large collection of these records plus an index that makes nearest-neighbor search fast.

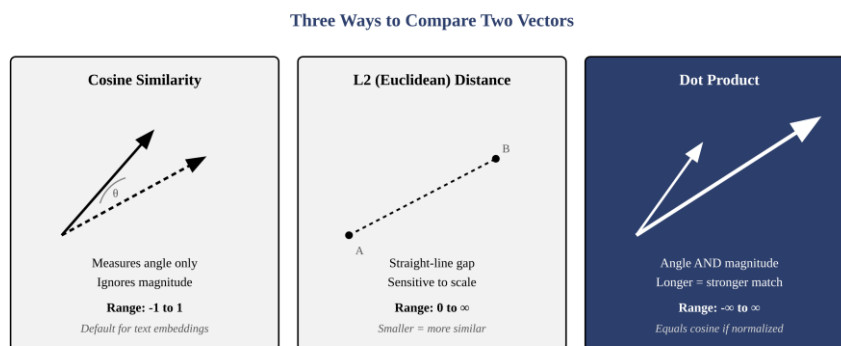
It helps to contrast this with the database you already know. A relational table answers WHERE id = 5 with an index lookup that either matches or does not — there is no concept of "close enough." A vector database answers a fundamentally different question: "of everything I have stored, which k records have embeddings nearest to this one?" That is a similarity search, not an exact match, and it requires a different kind of index entirely — the approximate-nearest-neighbor structures covered briefly in this chapter and in more depth wherever your chosen vendor documents its internals.

The query path in one sentence

A query text is embedded with the same model used at ingestion time, the resulting vector is compared against every stored embedding (or, in practice, against an approximate index of them), and the store returns the `n_results` closest record IDs along with their documents, metadata, and a distance score — that score, and how to interpret it, is the subject of the next section.

9.2 Cosine Similarity, L2 Distance, Dot Product — Pick One

Every vector database needs a way to score "how close are these two vectors," and every vendor exposes a choice between essentially the same three options. They are not interchangeable, and picking the wrong one for your embedding model silently degrades retrieval quality without throwing a single error — which makes this one of the easier mistakes to make and the hardest to notice.



Rule of thumb: normalized embeddings $\Rightarrow$ cosine and dot product rank identically.

Figure 9.2 — The three distance metrics and what each one is actually sensitive to.

Figure 9.2 — The three distance metrics and what each one is actually sensitive to.

Cosine similarity

Cosine similarity measures the angle between two vectors and ignores their magnitude entirely. Two vectors pointing in exactly the same direction score 1, perpendicular vectors score 0, and opposite vectors score -1 . For text embeddings, where the direction of the vector encodes meaning and magnitude is mostly an artifact of how the encoder happened to scale its output, this is almost always the right choice. It is the default in the `VectorStore` class used throughout this book, configured explicitly via the collection's `hnsw:space` metadata key.

L2 (Euclidean) distance

L2 distance is the literal straight-line distance between two points in the embedding space — the same formula you used for distance between two points on a graph in school, just extended to 384 dimensions. Smaller is better, and the range is unbounded: 0 means identical, and the number grows without limit as vectors diverge. Because it is sensitive to magnitude, two embeddings that point in nearly the same direction but were produced with different scaling can register as far apart, which is rarely what you want for semantic text search. L2 is more common in domains where absolute scale genuinely carries information — image embeddings or certain recommendation systems — and less common for sentence-transformer output.

Dot product

The dot product combines angle and magnitude into a single number: it rewards vectors that point the same way and rewards them more if they are also long. For raw, unnormalized embeddings this introduces a confound — a vector might score higher purely because the encoder gave it a larger magnitude, not because it is semantically closer to the query. There is, however, an important special case that makes dot product attractive in practice.

Definition — Normalized embedding

An embedding vector scaled so that its L2 norm equals 1.0 — written $\|v\| = 1.0$. Chapter 8 covered why sentence-transformers normalizes its output by default. Once every vector in a collection has unit length, dot product and cosine similarity rank results identically, because the magnitude term that distinguishes them has been fixed to exactly 1 for every vector.

This is why many production vector databases default to dot product internally even when the user-facing setting says "cosine" — if embeddings are pre-normalized, computing a dot product is cheaper than computing a full cosine similarity (which requires two extra division operations per comparison), and the ranking is mathematically identical. The `EmbeddingManager` class built in Chapter 8 calls `normalize_embeddings=True` on every `encode()` call specifically so this equivalence holds.

Common pitfall — Mismatched metric and embedding model

Switching embedding models without checking whether the new model's output is normalized — or without updating the collection's distance metric to match — produces retrieval results that look plausible but are subtly wrong. A collection created with `hnsw:space: "cosine"` will still run and still return results if you swap in a model whose vectors are not unit-length; it simply will not rank them correctly. Always re-verify normalization and metric choice together whenever the embedding model changes, which Chapter 9's migration discipline in Section 9.8 is built around.

The practical rule for this book

Use cosine similarity. Ensure embeddings are L2-normalized at generation time (Chapter 8 already does this). Every collection created by the `VectorStore` class in this chapter sets `hnsw:space` to "cosine" explicitly in the collection metadata, rather than relying on a vendor default that may differ between local development and a hosted alternative.

9.3 ChromaDB for Local Development

ChromaDB is the vector store used throughout this book, and the reason is almost entirely about friction. It runs in-process — there is no server to start, no Docker container to manage, no network port to open — and it persists its index to a directory on disk with a single line of configuration. For a book project, a prototype, or a small-to-medium production deployment running on one machine, that combination of zero infrastructure and durable storage is hard to beat.

Analogy — ChromaDB as a notebook, not a filing cabinet warehouse

A hosted vector database like Pinecone is a professional filing-cabinet warehouse: someone else maintains the building, the climate control, and the security guards, and you pay rent per cabinet. ChromaDB is a notebook you keep on your own desk — you write directly into it, it is always exactly where you left it, and there is no one else to coordinate with. A notebook does not scale to a warehouse's volume, but for one person's desk, it is faster to reach for and costs nothing to open.

Getting started requires installing the package and creating a persistent client pointed at a directory:

```
import chromadb

client = chromadb.PersistentClient(path="../data/vector_store")
collection = client.get_or_create_collection(
    name="documents",
    metadata={
        "Description": "Collection of document embeddings for RAG
system.",
        "hnsw:space": "cosine",
    },
)
print(collection.count())
```

Two details here are easy to skim past and important not to. First, `get_or_create_collection` means this code is safe to run repeatedly — the first call creates the collection, every subsequent call simply attaches to the existing one, which is what makes a script like `ingest.py` idempotent with respect to collection setup. Second, the `hnsw:space` metadata key is where the distance metric from Section 9.2 actually gets configured; ChromaDB's default differs by version, so setting it explicitly is the safer habit.

What ChromaDB is doing under the hood

ChromaDB uses HNSW (Hierarchical Navigable Small World), the same graph-based approximate-nearest-neighbor algorithm that underlies most modern vector databases, including Pinecone, Weaviate, and Qdrant. Rather than comparing a query vector against every stored vector — an operation that scales linearly and becomes slow past a few hundred thousand records — HNSW builds a multi-layer graph where each vector is connected to a handful of its nearest neighbors. A query then navigates that graph, starting at a coarse top layer and descending into finer layers, reaching a near-optimal answer in a number of comparisons that

grows logarithmically rather than linearly with collection size. This is why a query against a collection of three thousand chunks and a query against three hundred thousand chunks can feel similarly fast — at the cost of returning approximate, not provably exact, nearest neighbors, a trade-off virtually every production retrieval system accepts willingly.

Where ChromaDB stores its data

A persistent ChromaDB store is, physically, a directory containing a `chroma.sqlite3` file — which holds document text, metadata, and collection bookkeeping — alongside one subdirectory per collection (named with an auto-generated UUID) holding the binary HNSW index files. Both pieces are required; deleting either one breaks the store. This detail matters directly for Section 9.6 on persistence and Section 9.8 on backups, since a correct backup has to capture the whole directory, not just the parts that look like "the database."

When ChromaDB stops being the right choice

ChromaDB's single-process model is also its ceiling. It is not built for multiple machines writing to the same store concurrently, it does not offer the operational tooling (replication, automatic backups, access control) that a managed service provides out of the box, and its performance, while good, has not been tuned for billion-vector scale the way some purpose-built distributed systems have. None of that matters for this book's project or for most personal and small-team tools — but it is worth knowing the boundary before you hit it. Section 9.4 surveys the alternatives and the specific signal that should make you reach for one of them.

9.4 Alternatives: FAISS, Pinecone, Weaviate, Qdrant, Milvus, Typesense — When to Use Which

ChromaDB is the right default for this book, but it is not the only vector store you will encounter, and each alternative exists because it makes a different trade-off that ChromaDB does not. The table below summarizes the practical differences; the decision diagram that follows it turns those differences into a path you can actually follow.

Store	Deployment model	Best for	Trade-off
FAISS	Embedded library, no server	Maximum control, custom indexing, research	You build persistence, filtering, and the API layer yourself
Pinecone	Fully managed SaaS	Teams that want zero ops and instant scale	Per-vector hosting cost; data leaves your infrastructure
Weaviate	Self-hosted or managed cloud	Rich schema, hybrid search, GraphQL-style queries	Heavier to operate than ChromaDB; more moving parts
Qdrant	Self-hosted or managed cloud	Strong filtering performance, Rust-speed search	Smaller ecosystem of integrations than Pinecone/Weaviate
Milvus	Self-hosted distributed cluster	Billion-scale vectors, horizontal scaling	Operationally heavy — needs etcd, object storage, and a cluster
Typesense	Self-hosted or managed	Search-first products needing keyword + vector hybrid	Vector search is a feature of a search engine, not the core product

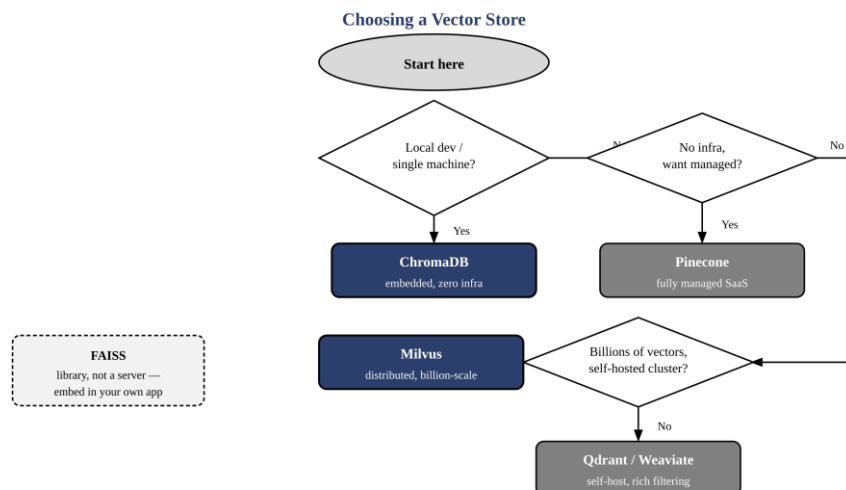


Figure 9.3 — A practical decision path through the vector-store landscape.

Figure 9.3 — A practical decision path through the vector-store landscape.

FAISS — a library, not a database

FAISS (Facebook AI Similarity Search) is conceptually different from the rest of this list: it is a C++/Python library you import and call, not a server you connect to. It has no built-in persistence format you don't write yourself, no metadata filtering, and no concept of a collection — you get an extremely fast and well-researched implementation of approximate-nearest-neighbor search and nothing else. This makes FAISS the right tool when you are building your own retrieval infrastructure from first principles, doing algorithm research, or need an index type ChromaDB doesn't expose (such as product-quantized indexes for billion-scale, memory-constrained search). For an application that wants persistence, metadata filtering, and an HTTP or Python client out of the box, the bookkeeping FAISS leaves you to build is exactly what ChromaDB already provides.

Pinecone — managed, opinionated, and infrastructure-free

Pinecone trades control for convenience: you create an index through an API call, push vectors to it, and never think about servers, disks, or HNSW parameters again. This is the right choice when a team does not want to operate any infrastructure at all, needs to scale from thousands to tens of millions of vectors without re-architecting, or is building a product where vector search is a small piece of a much larger system and the team's attention belongs elsewhere. The cost is real, both financially (per-vector and per-query pricing) and in terms of data residency — your embeddings live on Pinecone's infrastructure, which matters for some compliance contexts.

Weaviate and Qdrant — self-hosted middle ground

Weaviate and Qdrant occupy similar territory: both can be self-hosted (via Docker) or used as a managed cloud offering, both support rich metadata filtering and hybrid (vector plus keyword) search, and both are reasonable choices once a project outgrows a single Python process but is not yet at the scale that justifies Milvus's operational complexity. Weaviate leans toward a schema-first, GraphQL-flavored API and built-in support for multiple embedding providers; Qdrant leans toward raw query performance and a simpler, more Pythonic API surface. Choosing between them is usually a matter of which API style your team prefers rather than a hard technical constraint.

Milvus — when the dataset truly does not fit on one machine

Milvus is built for the scale at which a single-machine HNSW index stops being practical — hundreds of millions to billions of vectors — by distributing storage and compute across a cluster, typically backed by etcd for coordination and an object store (S3 or similar) for persistence. That capability comes with real operational weight: running Milvus well means running a small distributed system, not a single container. Reach for it when the vector count, not the query rate, is the bottleneck.

Typesense — vector search as one feature among several

Typesense is, first, a fast typo-tolerant keyword search engine; vector search was added as a capability on top of that foundation rather than being the product's reason for existing. It is a strong choice when the application genuinely needs hybrid search — exact product names, SKUs, or proper nouns alongside semantic similarity — and a weaker choice as a pure vector store for a RAG pipeline where keyword search is not a requirement.

Common pitfall — Picking a vendor before you have a load profile

It is tempting to reach for Pinecone or Milvus early because they sound more "production-grade" than a local file. In practice, the migration path from ChromaDB to a hosted or distributed store is straightforward — re-embed once, or even just export and re-import the same vectors — so there is little cost to starting local and almost no cost paid later for having done so. The expensive mistake is the opposite one: standing up a distributed cluster for a corpus that would have fit comfortably in a ChromaDB directory on a laptop.

9.5 Collections, IDs, Metadata Filters

A single ChromaDB client can manage many independent collections — separate namespaces, each with its own vectors, its own dimensionality, and its own distance metric. Within a collection, every record needs a unique ID and may carry a metadata dictionary; that metadata is what makes it possible to narrow a similarity search to a meaningful subset before distance is even computed.

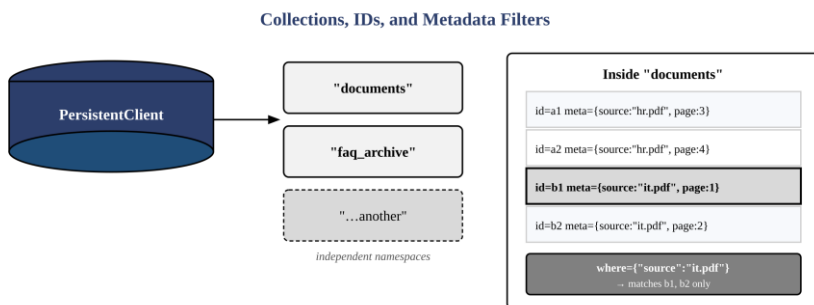


Figure 9.4 — Collections partition the store; metadata filters narrow a query to a subset of one collection.

Figure 9.4 — Collections partition the store; metadata filters narrow a query to a subset of one collection.

Collections

Think of a collection the way you would think of a table in a relational database, with one important difference: every vector inside a collection must share the same embedding dimensionality, because the HNSW index built for that collection is structured around a fixed vector size. A project that embeds both short FAQ entries and long technical documents with different embedding models needs two collections, not one — mixing 384-dimensional and 768-dimensional vectors in the same collection is not supported and will fail at insertion time.

```
# Two independent collections, same client
docs = client.get_or_create_collection("documents")
faqs = client.get_or_create_collection("faq_archive")

# Each behaves as a separate namespace:
docs.count()    # vectors in 'documents' only
faqs.count()    # vectors in 'faq_archive' only
```

IDs

ChromaDB requires every record to have a string ID, and it is the caller's responsibility to generate one — there is no auto-increment. Two properties matter when designing an ID scheme: IDs must be unique within a collection (inserting a duplicate ID overwrites the existing record rather than raising an error, which is a useful property for idempotent re-ingestion and a dangerous one if unintended), and IDs should ideally be stable across re-runs if you want repeated ingestion of the same source files to update rather than duplicate entries — a concern Chapter 10 addresses directly.

The `VectorStore` class built in Section 9.7 generates IDs by combining a fresh UUID with the position of the chunk inside its batch:

```
import uuid

doc_id = str(uuid.uuid4()) + f"_{i}"
```

Metadata

Metadata is a flat dictionary attached to each record. ChromaDB restricts metadata values to strings, integers, floats, and booleans — there is no native support for nested dictionaries or arbitrary objects. This is a real constraint, not a documentation oversight, and it shows up the moment a loader hands you metadata containing a list or a nested structure (common with PDF loaders that attach page-layout information). The sanitization helper used in this book's `VectorStore` class handles this by serializing anything that is not a primitive type to a JSON string before insertion:

```
def _sanitize_metadata_value(self, value):
    if isinstance(value, (str, int, float, bool)) or value is None:
        return value
    if isinstance(value, list):
        return [
            item if isinstance(item, (str, int, float, bool)) or item is
            None
            else json.dumps(item, ensure_ascii=True)
            for item in value
        ]
    try:
        return json.dumps(value, ensure_ascii=True)
    except Exception:
        return str(value)
```

Metadata filters

A query can pass a where clause that ChromaDB applies before — not after — the similarity search, restricting the candidate pool to records whose metadata matches exactly:

```
results = collection.query(  
    query_embeddings=[query_vector],  
    n_results=5,  
    where={"source": "it_policy.pdf"},  
)
```

This is exact-match filtering, and that single word — exact — is the source of one of the more common silent failures in a RAG system built on metadata filters.

Common pitfall — Free-form metadata values break filters silently

ChromaDB metadata filters are exact-match. "Policy", "policy", and "policies" are three distinct values, and a where={"doc_type": "policy"} filter against records tagged "Policy" returns nothing — not an error, just an empty result set that looks identical to "there is no matching content." Normalize metadata values (lower-case, fixed vocabulary, an enum if your loader allows it) at ingestion time, before they ever reach the vector store, rather than trying to catch the mismatch at query time.

9.6 Persistence: Saving the Store to Disk and Reloading

Everything in this chapter so far has assumed the vector store survives the Python process that created it — that you can ingest a corpus today, shut your laptop, and query the same data tomorrow without re-embedding anything. That property is persistence, and ChromaDB's `PersistentClient` provides it by writing every insertion to disk immediately, rather than holding the collection only in memory.

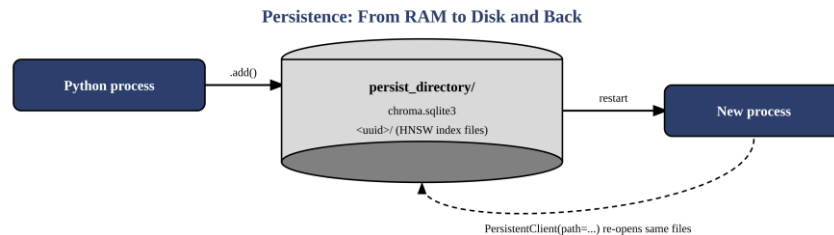


Figure 9.5 — The on-disk layout survives process restarts; a new client at the same path sees the same data.

Figure 9.5 — The on-disk layout survives process restarts; a new client at the same path sees the same data.

The mechanism is almost embarrassingly simple from the calling code's perspective: point a new `PersistentClient` at the same directory path used previously, and the existing collections, with all their records, are immediately available.

```
import chromadb

# Process A – ingestion run
client = chromadb.PersistentClient(path="../data/vector_store")
collection = client.get_or_create_collection("documents")
collection.add(ids=[...], embeddings=[...], documents=[...])
# process exits here – data is already on disk, no explicit save()
needed

# Process B – started later, possibly days later
client = chromadb.PersistentClient(path="../data/vector_store")
collection = client.get_or_create_collection("documents")
print(collection.count()) # reflects everything Process A inserted
```

Two implementation details are worth internalizing rather than treating as magic. First, there is no separate `save()` call to remember — every `add()` call writes through to disk as part of the call itself, which means a crash immediately after an insertion does not lose that insertion (though it could, in principle, leave an in-progress batch partially written, which is one reason Section 9.7 builds batched inserts deliberately rather than one giant call). Second, `get_or_create_collection` is doing real work on reload: it inspects the existing on-disk metadata, confirms the collection exists, and attaches to it rather than creating a fresh, empty collection with the same name — which is why this call is safe to make at the start of every script that touches the store.

Analogy — PersistentClient as a USB drive, not RAM

An in-memory-only client (`chromadb.Client()` with no path) behaves like RAM — fast, but everything disappears the instant the process ends. `PersistentClient` behaves like a USB drive: every write lands on physical storage immediately, and plugging the same drive into a different computer (a different Python process, possibly on a different day) gives you back exactly what was last written to it.

Verifying persistence actually worked

Because persistence failures are silent — a misconfigured path simply creates a new, empty store rather than raising an error — it is worth building the habit of checking `collection.count()` immediately after opening a client, especially in a fresh script or notebook session. A count of zero where you expected thousands of records is the single most common symptom of pointing `PersistentClient` at the wrong directory, and catching it at that line is far cheaper than debugging an empty-looking retrieval pipeline three files downstream.

9.7 Building the VectorStore Class with Batched Inserts

Everything covered so far in this chapter — record structure, distance metrics, collections, metadata, persistence — comes together in a single class: `VectorStore`. This is the class that the ingestion pipeline in Chapter 10 and the retriever in Chapter 11 will both depend on, so it is worth building it carefully and understanding exactly why each method is shaped the way it is.

Initialization and the persistent collection

The constructor opens (or creates) a persistent client and collection, exactly as shown in Sections 9.3 and 9.6, with the distance metric pinned explicitly so the collection's behavior does not depend on whichever default the installed ChromaDB version happens to ship with:

```
import json
import os
import uuid
from typing import Any, Dict, List

import numpy as np
import torch
import chromadb

class VectorStore:
    def __init__(
        self,
        collection_name: str = "documents",
        persist_directory: str = "../data/vector_store",
    ):
        self.collection_name = collection_name
        self.persist_directory = persist_directory
        self.client = None
        self.collection = None
        self._initialize_store()

    def _initialize_store(self):
        try:
            os.makedirs(self.persist_directory, exist_ok=True)
            self.client = chromadb.PersistentClient(path=self.persist_directory)
            self.collection = self.client.get_or_create_collection(
                name=self.collection_name,
                metadata={
                    "Description": "Collection of document embeddings for RAG system.",
                    "hnsw:space": "cosine",
                },
            )
            print(
                f"Vector store initialized: "
                f"collection='{self.collection_name}', "
                f"path='{self.persist_directory}'"
            )
            print(f"Existing documents in collection: {self.collection.count()}")
        except Exception as e:
            print(f"Error initializing vector store: {e}")
            raise
```

Notice the `os.makedirs(..., exist_ok=True)` call before the client is created — ChromaDB will not create a missing parent directory on its own, and failing to ensure it exists first is a quiet way to get a confusing error on a machine where the data directory has never been created.

Metadata sanitization

Before any record reaches ChromaDB, its metadata dictionary passes through the sanitization helper introduced in Section 9.5, applied key-by-key so that a single malformed field does not reject an otherwise valid insertion:

```
def _sanitize_metadata_value(self, value):
    if isinstance(value, (str, int, float, bool)) or value is None:
        return value
    if isinstance(value, list):
        return [
            item if isinstance(item, (str, int, float, bool)) or
item is None
            else json.dumps(item, ensure_ascii=True)
            for item in value
        ]
    try:
        return json.dumps(value, ensure_ascii=True)
    except Exception:
        return str(value)

def _sanitize_metadata(self, metadata: Dict[str, Any]) -> Dict[str,
Any]:
    return {str(k): self._sanitize_metadata_value(v) for k, v in
metadata.items() }
```

Inserting a single batch

`add_document()` takes a list of LangChain Document objects and a matching array of embeddings, and writes them through to the collection as one ChromaDB `add()` call. This is the unit of work that every higher-level batching method ultimately calls:

```
def add_document(self, documents: List[Any], embeddings:
np.ndarray):
    if len(documents) != len(embeddings):
        raise ValueError("Number of documents and embeddings must
match.")
    print(f"Adding {len(documents)} documents to the vector
store...")

    ids, metadatas, documents_text, embeddings_list = [], [], [], []

    for i, (doc, embedding) in enumerate(zip(documents,
embeddings)):
        ids.append(str(uuid.uuid4()) + f"_{i}")
        metadata = {
            **dict(doc.metadata),
            "doc_index": i,
            "content_length": len(doc.page_content),
        }
        metadatas.append(self._sanitize_metadata(metadata))
        documents_text.append(doc.page_content)
        embeddings_list.append(embedding.tolist())

    self.collection.add(
        ids=ids,
        embeddings=embeddings_list,
        metadatas=metadatas,
        documents=documents_text,
    )
    print(f"{len(documents)} documents added to vector store.")
```

Common pitfall — Calling `.tolist()` inside the loop, not before it

ChromaDB's client expects embeddings as plain Python lists, not numpy arrays — passing a numpy array directly either fails or silently serializes incorrectly depending on the client version. The `embedding.tolist()` call above is doing necessary type conversion, not a stylistic choice; dropping it is a common source of a confusing insertion-time error that has nothing to do with the data itself.

Why a single bulk insert does not scale

It is tempting to embed an entire corpus in one call and pass the whole result to `add_document()` in one shot. For a few hundred chunks this works fine. Past a few thousand, two separate problems appear at once: the embedding model, if running on GPU, can exhaust VRAM trying to encode the entire corpus in a single forward pass, and a single oversized `add()` call holds the full embeddings array in memory for the duration of the write, multiplying peak memory use right at the moment a crash is most costly to recover from.

The fix is batching: slice the corpus into fixed-size chunks, embed and insert one slice at a time, and release GPU memory between slices.

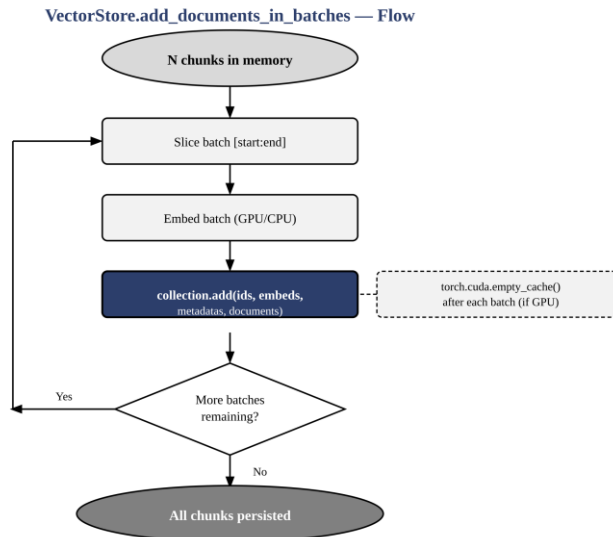


Figure 9.6 — One write per batch keeps memory bounded and avoids a single oversized insert call.

```
def add_documents_in_batches(
    self,
    pdf_chunks,
    embedding_manager,
    batch_size: int = 128,
):
    total = len(pdf_chunks)
    print(f"Adding {total} chunks in batches of {batch_size}")

    for start in range(0, total, batch_size):
        end = min(start + batch_size, total)
        batch_docs = pdf_chunks[start:end]
        batch_texts = [doc.page_content for doc in batch_docs]

        print(f"Processing batch {start} - {end - 1}")
        batch_embeddings = embedding_manager.generate_embedding(
            batch_texts, batch_size=batch_size
        )
        self.add_document(batch_docs, batch_embeddings)

        if torch.cuda.is_available():
            torch.cuda.empty_cache()

    print("All batches inserted successfully.")
```

This single change — batching embedding generation and insertion together, rather than embedding everything first and inserting second — is, in practice, the difference between a pipeline that completes reliably on a laptop and one that crashes a Jupyter kernel partway through a large corpus. A reasonable starting point is `batch_size=128` for a small sentence-transformer model on a modern GPU; on CPU-only

hardware, or with a larger embedding model, 16 to 32 is often a safer starting value, with room to increase once you have confirmed memory headroom.

Analogy — Batched inserts as moving house with a hand truck

Carrying every box from a house to a truck in one trip — stacked precariously, all at once — is the unbatched approach: it works for a studio apartment and collapses under its own weight for anything bigger. A hand truck carrying a few boxes per trip is slower per trip but finishes the whole house reliably, because no single load ever exceeds what the truck (or the GPU) can actually bear.

9.8 Backups, Migrations, and Versioning Your Index

Everything up to this point treats the vector store as something you build once and query forever. In practice an index is never really finished — source documents get updated, a better embedding model is released, a chunking strategy turns out to have been a mistake — and each of those events requires touching a store that production traffic may currently depend on. This section covers the discipline that keeps those changes from turning into outages or silent data loss.

Backups

Because a ChromaDB store is just a directory on disk (Section 9.3), backing it up is, mechanically, no different from backing up any other directory of files — but it is easy to under-scope the backup if you only think of "the database" as the `chroma.sqlite3` file and forget the per-collection HNSW index subdirectories that sit alongside it.

```
# Simple, cron-friendly backup of an entire ChromaDB store
tar -czf backups/vector_store_$(date +%Y-%m-%d).tar.gz \
    -C ../data vector_store

# Restore: unpack into a fresh path and point a new client at it
tar -xzf backups/vector_store_2026-06-20.tar.gz -C /tmp/restored
```

A few habits make this reliable rather than theoretical: take the snapshot while no write is in progress (or accept that a snapshot mid-write may capture a partially completed batch, which batched inserts in Section 9.7 make less catastrophic but not impossible), keep more than one snapshot generation so a corrupted backup is not your only backup, and periodically test the restore path itself — a backup that has never been restored is a backup you do not actually know works.

Common pitfall — Backing up only the .sqlite3 file

Copying `chroma.sqlite3` alone preserves document text and metadata but not the HNSW index binaries stored in the per-collection UUID subdirectory next to it. A restore from that partial backup will open without error and report the correct `collection.count()`, but every similarity search against it will fail or behave incorrectly, because the index ChromaDB needs to actually perform nearest-neighbor search is missing. Back up the whole `persist_directory`, not the file that happens to look most like "the database."

Migrations

A migration is any change that requires re-deriving the vectors themselves — most commonly, switching embedding models. Because a collection's vectors must share one dimensionality and were produced by one

model's notion of semantic space, you cannot incrementally update a live collection from a 384-dimensional model to a 768-dimensional one; the only correct path is to build a new collection from scratch and cut traffic over once it is validated.

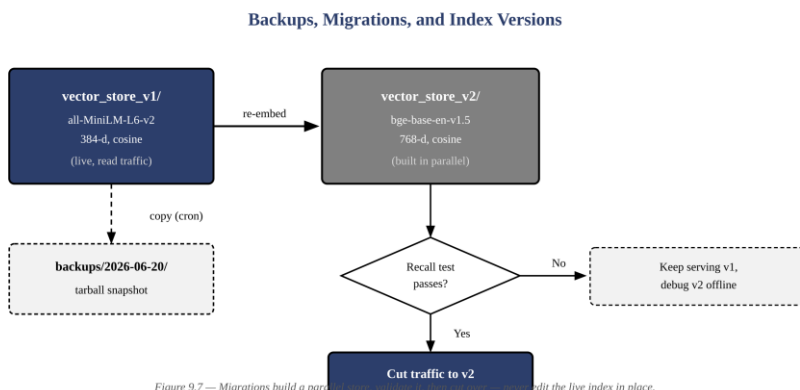


Figure 9.7 — Migrations build a parallel store, validate it, then cut over — never edit the live index in place.

```

# Build the new store in parallel, without touching the live one
old_store = VectorStore(
    collection_name="documents",
    persist_directory="../data/vector_store_v1",
)
new_store = VectorStore(
    collection_name="documents",
    persist_directory="../data/vector_store_v2",
)

# Re-embed the same source chunks with the new model
new_embedding_manager = EmbeddingManager(model_name="BAAI/bge-base-en-
v1.5")
new_store.add_documents_in_batches(pdf_chunks, new_embedding_manager)

# Only after validation (see below) does application config switch
# VECTOR_STORE_PATH from vector_store_v1 to vector_store_v2.

```

The validation step in the middle of that flow is not optional. Before cutting traffic over, run a small, fixed set of known queries against both the old and new store and compare the retrieved chunks — a new embedding model that retrieves obviously worse results for queries you already understand is far cheaper to catch here than after it is serving production answers.

Definition — Cutover

The moment a system's read traffic switches from an old data source to a newly validated replacement — here, from `vector_store_v1` to `vector_store_v2` — typically achieved by changing a single configuration value (a path or collection name) rather than by mutating the old store in place. A clean cutover means the old store still exists, untouched, as an instant rollback target if the new one misbehaves under real traffic.

Versioning

The directory-naming convention in the example above — `vector_store_v1`, `vector_store_v2` — is the simplest form of versioning, and for a project of this book's scale it is sufficient. The discipline it encodes is the part worth keeping regardless of scale: never edit a live, serving index in place. Every structural change (new embedding model, new chunking parameters, new metadata schema) becomes a new, separately named store, built and validated alongside the old one, with the old one kept around until the new one has proven itself under real queries. That single rule — build beside, validate, then cut over, never mutate in place — is what turns a vector database from something fragile into something you can change your mind about safely.